8.2 – Incompatibilities in the Libraries

- Parameters for the garbage collection are not set with the options "`incremental`" and "`generational`"; instead, there is a new option "`param`" to that end. Moreover, there were some changes in the parameters themselves.

8.3 – Incompatibilities in the API

- In `lua_call` and related functions, the maximum value for the number of required results (`nresults`) is 250. If you really need a larger value, use `LUA_MULTRET` and then adjust the stack size. Previously, this limit was unspecified.
- `lua_newstate` has a third parameter, a seed for the hashing of strings.
- The function `lua_resetthread` is deprecated; it is equivalent to `lua_closethread` with `from` being `NULL`.
- The function `lua_setcstacklimit` is deprecated. Calls to it can simply be removed.
- The function `lua_dump` changed the way it keeps the stack through the calls to the writer function. (That was not specified in previous versions.) Also, it calls the writer function one extra time, to signal the end of the dump.
- Parameters for the garbage collection are not set with the options `LUA_GCINC` and `LUA_GCGEN`; instead, there is a new option `LUA_GCPARAM` to that end. Moreover, there were some changes in the parameters themselves.
- The function `lua_pushvfstring` now reports errors, instead of raising them.

9 – The Complete Syntax of Lua

Here is the complete syntax of Lua in extended BNF. As usual in extended BNF, {A} means 0 or more As, and [A] means an optional A. (For operator precedences, see §3.4.8; for a description of the terminals Name, Numeral, and LiteralString, see §3.1.)

```
chunk ::= block

block ::= {stat} [retstat]

stat ::=  ‘;’ |
         varlist ‘=’ explist |
         functioncall |
         label |
         break |
         goto Name |
         do block end |
         while exp do block end |
         repeat block until exp |
         if exp then block {elseif exp then block} [else block] end |
         for Name ‘=’ exp ‘,’ exp [‘,’ exp] do block end |
         for namelist in explist do block end |
         function funcname funcbody |
         local function Name funcbody |
         global function Name funcbody |
```

```
        local attnamelist ['=' explist] |
        global attnamelist ['=' explist] |
        global [attrib] '*'

attnamelist ::=  [attrib] Name [attrib] {',' Name [attrib]}

attrib ::= '<' Name '>'

retstat ::= return [explist] [';']

label ::= '::' Name '::'

funcname ::= Name {'.' Name} [':' Name]

varlist ::= var {',' var}

var ::=  Name | prefixexp '[' exp ']' | prefixexp '.' Name

namelist ::= Name {',' Name}

explist ::= exp {',' exp}

exp ::=  nil | false | true | Numeral | LiteralString | '...' | functiondef |
         prefixexp | tableconstructor | exp binop exp | unop exp

prefixexp ::= var | functioncall | '(' exp ')'

functioncall ::=  prefixexp args | prefixexp ':' Name args

args ::=  '(' [explist] ')' | tableconstructor | LiteralString

functiondef ::= function funcbody

funcbody ::= '(' [parlist] ')' block end

parlist ::= namelist [',' varargparam] | varargparam

varargparam ::= '...' [Name]

tableconstructor ::= '{' [fieldlist] '}'

fieldlist ::= field {fieldsep field} [fieldsep]

field ::= '[' exp ']' '=' exp | Name '=' exp | exp

fieldsep ::= ',' | ';'

binop ::=  '+' | '-' | '*' | '/' | '//' | '^' | '%' |
         '&' | '~' | '|' | '>>' | '<<' | '..' |
         '<' | '<=' | '>' | '>=' | '==' | '~=' |
         and | or
```